

1. 五子棋

游戏流程

- 1. 进入游戏：输入1
- 2. 游戏界面：打印回合数、棋盘、提示输入落子坐标

```
College\Check\"game
----- 游戏主菜单 请选择 -----
-----      1. 进入游戏      -----
-----      2. 进入设置      -----
-----      3. 退出游戏      -----
      请输入你的选择：1
当前回合数：0
-----棋盘-----
      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
0 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
1 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
2 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
3 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
4 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
5 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
6 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
7 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
8 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
9 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
10 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
11 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
12 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
13 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
14 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
15 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
16 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
17 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
18 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
      请输入落子坐标(格式：x y，按回车确认):
█
```

- 3. 第 7 回合的棋盘
- 4. 棋盘中 空白处为 空地，X 表示 黑子，O 表示 白子

```
请输入落子坐标(格式: x y, 按回车确认):
11 8
落子成功
当前回合数: 7
-----棋盘-----
   0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
0 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
1 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
2 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
3 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
4 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
5 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
6 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
7 |  |  |  |  |  |  |  |  |  O |  |  |  |  |  |  |  |  |
8 |  |  |  |  |  |  |  |  |  X |  |  |  |  |  |  |  |  |
9 |  |  |  |  |  |  |  |  |  X |  O |  |  |  |  |  |  |  |
10|  |  |  |  |  |  |  |  |  X |  |  O |  |  |  |  |  |  |
11|  |  |  |  |  |  |  |  |  X |  |  |  |  |  |  |  |  |
12|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
13|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
14|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
15|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
16|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
17|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
18|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
请输入落子坐标(格式: x y, 按回车确认):
```

- 5. 胜利界面：打印回合数，棋盘，以及获胜方。
- 6. 提示按任意键继续。

```
当前回合数: 10
-----棋盘-----
   0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
0 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
1 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
2 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
3 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
4 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
5 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
6 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
7 |  |  |  |  |  |  |  |  |  O |  |  |  |  |  |  |  |  |
8 |  |  |  |  |  |  |  |  |  X |  |  |  |  |  |  |  |  |
9 |  |  |  |  |  |  |  |  X |  X |  O |  |  |  |  |  |  |  |
10|  |  |  |  |  |  |  |  |  X |  O |  O |  |  |  |  |  |  |
11|  |  |  |  |  |  |  |  |  X |  O |  |  |  |  |  |  |  |
12|  |  |  |  |  |  |  |  |  X |  |  |  |  |  |  |  |  |
13|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
14|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
15|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
16|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
17|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
18|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
黑棋玩家获胜!
请按任意键继续...
```

按下任意键可以回到主菜单选择继续游戏 或 退出游戏

头文件

```

1  #include <iostream>
2  #include <iomanip>
3  #include <cstdlib>
4  #include <vector>
5
6  using namespace std;

```

数据设计

```

1  // C/C++:
2  // ----- 数据设计 -----
3  /*
4      棋盘:
5          map[i][j]表示坐标(i,j)的值
6          0表示空地
7          1表示黑子
8          2表示白子
9      如: map[3][6] = 1 表示(3,6)的位置是黑子
10 */
11 int map[19][19];
12
13 // map的行数和列数
14 const int map_row = sizeof(map) / sizeof(map[0]);
15 const int map_col = sizeof(map[0]) / sizeof(map[0][0]);
16
17 // 表示当前回合数 偶数表示黑棋落子 奇数表示白棋落子
18 // 如: flag = 20 表示当前是第[20]次落子 由黑方落子
19 int flag;
20
21 // ----- 数据设计 -----

```

Service

```

1  // ----- service -----
2  /*
3      负责人: Rain
4      功能: init: 初始化游戏数据
5          将棋盘的值初始化为0
6          当前回合设为黑棋(flag设为0)
7      参数: void
8      返回值: void
9  */
10 void init();
11
12 /*
13     *难点1
14     负责人: Rain
15     功能: iswin: 根据传入的坐标(map对应位置)和flag值 判断落点后是否获胜
16     参数:
17         x: 当前回合落子的x坐标
18         y: 当前回合落子的y坐标
19     返回值:
20         0表示没有获胜
21         1表示黑子胜利
22         2表示白子胜利

```

```

23  */
24  int iswin(int x, int y);
25
26  /*
27      负责人: Rain
28      功能: dfs: 根据输入的map和起始坐标进行深度优先搜索 以及 连子计数
29      参数:
30          map: 棋盘
31          cur_row: 当前位置x坐标
32          cur_col: 当前位置y坐标
33          visit: 访问数组
34          prevNode: 当前棋子颜色
35          win_cnt: 连子计数
36          direction: 方向
37      返回值:
38          void
39  */
40  void dfs(int (&map)[map_row][map_col], int cur_row, int cur_col,
41      vector<vector<bool>> &visit, int prevNode, int &win_cnt, int direction);
42
43  /*
44      负责人: Rain
45      功能: playerMove: 在指定位置落子
46          如果map[x][y]是空地 则修改map[x][y]的值:改为相应颜色(flag对应颜色)
47      否则不操作
48      参数:
49          x: 当前回合落子的x坐标
50          y: 当前回合落子的y坐标
51      返回值:
52          0表示落子失败 (棋盘已经有子)
53          1表示落子成功
54  */
55  int playerMove(int x, int y);
56  // ----- service -----

```

View

```

1  // ----- view -----
2  /*
3      负责人: Rain
4      功能: menuView: 展示选项, 玩家可以在此选择进入游戏, 进入设置或退出游戏
5          进入游戏: 调用游戏界面函数gameView();
6          进入设置: 敬请期待...
7          退出游戏: 调用exit(0);
8      参数: void
9      返回值: void
10 */
11 void menuView();
12
13 /*
14     负责人: Rain
15     功能: gameView_ShowMap: 根据map数组 打印游戏棋盘
16     参数: void
17     返回值: void
18 */
19 void gameView_ShowMap();
20

```

```

21  /*
22     负责人: Rain
23     功能: winview: 根据flag的值 打印游戏胜利界面 用户可以按任意键回到主菜单
24     参数: void
25     返回值: void
26  */
27  void winview();
28
29  /*
30     *难点2
31     负责人: Rain
32     功能: gameview: 游戏界面整合
33         初始化游戏数据(调用函数init())
34         while(1){
35             打印游戏界面(调用函数gameview_ShowMap())
36             接收玩家坐标输入
37
38             落子(调用落子函数playerMove())
39             (如果落子失败 重新开始循环)
40
41             判断游戏是否胜利(调用胜利判断函数iswin())
42             (如果游戏胜利 调用胜利界面函数 然后结束当前界面)
43             切换玩家(修改flag值)
44         }
45     参数: void
46     返回值: void
47  */
48  void gameview();
49  // ----- view -----

```

主函数

```

1  int main()
2  {
3      menuview(); // 调用菜单界面函数
4      return 0;
5  }

```

函数定义

```

1  // 初始化游戏数据
2  void init()
3  {
4      // init map
5      for (auto &row : map) // 遍历棋盘的每一行
6      {
7          for (auto &element : row) // 遍历当前行的每一个元素
8          {
9              element = 0; // 将元素初始化为0, 表示空地
10         }
11     }
12
13     // init flag
14     flag = 0; // 初始化回合数为0, 表示黑棋先落子
15
16     return;

```

```

17 }
18
19 /*
20     思路：使用DFS算法 从落子点开始 向8个方向搜索 统计连子数量
21 */
22 // 判断当前落子是否获胜
23 int iswin(int x, int y)
24 {
25     // 返回值，默认棋子落子失败
26     int r_val = 0;
27
28     // 从落子点开始进行深度优先搜索
29     for(int dir = 1; dir <= 4; dir++)
30     {
31         // 访问标记
32         vector<vector<bool>> visit(map_row, vector<bool>(map_col, false));
33         // 连子计数
34         int win_cnt = 0;
35         dfs(map, x, y, visit, map[x][y], win_cnt, dir);
36
37         // 如果连子数量达到5，表示获胜
38         if(win_cnt >= 5)
39         {
40             r_val = map[x][y]; // 返回获胜方的棋子颜色
41             break;
42         }
43     }
44
45     return r_val;
46 }
47
48 // 深度优先搜索函数
49 void dfs(int (&map)[map_row][map_col], int cur_row, int cur_col,
50 vector<vector<bool>> &visit, int prevNode, int &win_cnt, int direction)
51 {
52     // 越界、当前位置不是同色棋子、已经访问过、是空地或者连子数量达到5，返回
53     if (cur_row < 0 || cur_col < 0 || cur_row == map_row || cur_col ==
54 map_col || map[cur_row][cur_col] != prevNode || visit[cur_row][cur_col] ||
55 prevNode == 0 || win_cnt == 5)
56     {
57         return ;
58     }
59
60     win_cnt++; // 连子数量加1
61     visit[cur_row][cur_col] = true; // 标记当前位置已访问
62
63     // 向8个方向进行深度优先搜索
64     if(direction == 1) // 上下搜索
65     {
66         dfs(map, cur_row + 1, cur_col, visit, map[cur_row][cur_col],
67 win_cnt, direction); // 向下
68         dfs(map, cur_row - 1, cur_col, visit, map[cur_row][cur_col],
69 win_cnt, direction); // 向上
70     }
71     else if(direction == 2) // 左右搜索
72     {
73         dfs(map, cur_row, cur_col + 1, visit, map[cur_row][cur_col],
74 win_cnt, direction); // 向右

```

```

69         dfs(map, cur_row, cur_col - 1, visit, map[cur_row][cur_col],
win_cnt, direction); // 向左
70     }
71     else if(direction == 3) // 左上右下搜索
72     {
73         dfs(map, cur_row - 1, cur_col - 1, visit, map[cur_row][cur_col],
win_cnt, direction); // 左上
74         dfs(map, cur_row + 1, cur_col + 1, visit, map[cur_row][cur_col],
win_cnt, direction); // 右下
75     }
76     else if(direction == 4) // 左下右上搜索
77     {
78         dfs(map, cur_row - 1, cur_col + 1, visit, map[cur_row][cur_col],
win_cnt, direction); // 右上
79         dfs(map, cur_row + 1, cur_col - 1, visit, map[cur_row][cur_col],
win_cnt, direction); // 左下
80     }
81
82     return ;
83 }
84
85 // 玩家落子函数
86 int playerMove(int x, int y)
87 {
88     // 落子结果，默认落子失败
89     int r_val = 0;
90
91     // 如果当前位置是空地
92     if (map[x][y] == 0)
93     {
94         // 根据flag的值确定落子颜色
95         map[x][y] = flag % 2 + 1;
96         r_val = 1; // 落子成功
97     }
98     // 棋盘上已有棋子，落子失败
99     else
100     {
101         r_val = 0;
102     }
103     return r_val;
104 }
105
106 // 菜单界面函数
107 void menuView()
108 {
109     // 循环显示菜单
110     while (true)
111     {
112         cout << "----- 游戏主菜单 请选择 -----" << endl;
113         cout << "-----      1. 进入游戏      -----" << endl;
114         cout << "-----      2. 进入设置      -----" << endl;
115         cout << "-----      3. 退出游戏      -----" << endl;
116         cout << "                      请输入你的选择: ";
117
118         // 接收用户输入
119         int select;
120         cin >> select;
121         switch (select)

```

```

122     {
123         // 进入游戏
124         case 1:
125             gameView();
126             break;
127
128         // 进入设置
129         case 2:
130             cout << "敬请期待" << endl;
131             break;
132
133         // 退出游戏
134         case 3:
135             exit(0);
136             break;
137
138         // 输入无效
139         default:
140             cout << "输入无效" << endl;
141             break;
142     }
143 }
144
145 return ;
146 }
147
148 // 显示游戏棋盘
149 void gameView_ShowMap()
150 {
151     // 显示当前回合数
152     cout << "当前回合数: " << flag << endl;
153
154     // 打印棋盘边界
155     cout << "-----棋盘-----"
156 << endl;
157     cout << "    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18"
158 << endl;
159     int i = 0;
160     for (auto &row : map) // 遍历棋盘的每一行
161     {
162         cout << setiosflags(ios::left) << setw(2) << i << "| "; // 打印行号
163         for (auto element : row) // 遍历当前行的每一个元素
164         {
165             cout << element << setiosflags(ios::left) << setw(2) << "|";
166         }
167         // 打印元素
168         cout << endl;
169         i++;
170     }
171
172     return ;
173 }
174
175 // 显示胜利界面
176 void winView()
177 {
178     // 根据flag的值判断获胜方
179     if ((flag) % 2 == 0)

```



```

177     {
178         gameview_ShowMap();
179         cout << "黑棋玩家获胜!" << endl;
180     }
181     else if ((flag) % 2 == 1)
182     {
183         gameview_ShowMap();
184         cout << "白棋玩家获胜!" << endl;
185     }
186     return ;
187 }
188
189 // 游戏界面整合函数
190 void gameView()
191 {
192     // 初始化游戏数据
193     init();
194
195     // 游戏循环
196     while(true)
197     {
198         int x, y;
199         gameview_ShowMap(); // 显示游戏棋盘
200
201         // 提示用户输入落子坐标
202         cout << "请输入落子坐标(格式: x y, 按回车确认): " << endl;
203         cin >> x >> y;
204
205         // 检查坐标合法性
206         if (x < 0 || x >= map_row || y < 0 || y >= map_col)
207         {
208             cout << "输入的坐标不合法, 请重新输入" << endl;
209             continue;
210         }
211
212         // 落子
213         if(playerMove(x, y) == 0)
214         {
215             cout << "落子失败, 请重新落子" << endl;
216             // 落子失败, 重新开始循环
217             continue;
218         }
219         // 落子成功
220         cout << "落子成功" << endl;
221
222         // 判断游戏是否胜利
223         if(iswin(x, y) == 1) // 黑子胜利
224         {
225             winView(); // 显示胜利界面
226             break;
227         }
228         else if(iswin(x, y) == 2) // 白子胜利
229         {
230             winView(); // 显示胜利界面
231             break;
232         }
233
234         // 游戏未结束, 继续循环

```

```

235         flag++; // 回合数加1
236     }
237     return ;
238 }

```

2. 问答题

1. 以下内容的代码实现, 由view还是service负责? (填写view或service)
 1. 游戏逻辑判断: **Service**
 2. 用户输入: **View**
 3. 用户输入数据合法判断/类型判断(比如区分输入是[方向]还是[空格]): **View**
 4. 界面展示: **View**
 5. 游戏数据修改: **Service**
2. 以下说法是否正确? (填写true或false)
 1. view有时需要调用service的功能 以实现逻辑判断和数据修改: **True**
 2. service有时需要调用view的功能 以实现界面展示: **False**
 3. service中可以写scanf和printf 用于接收用户输入和展示界面: **False**
 4. 涉及游戏数据判断和修改的, 大多在service实现: **True**
 5. 数据的设计不需要描述很清晰, 自己能看懂即可: **False**

3. 推箱子游戏

1. 开始界面: 可以选择: [选关界面]/[设置界面]/[排行榜界面]/退出游戏
2. 选关界面: 可以选择关卡进入[游戏界面], 或者回到[开始界面]
3. 游戏界面: 可以移动人物, 回退到[选关界面], 或者游戏胜利后展示[胜利界面]
4. 胜利界面: 提示游戏胜利, 并回到[选关界面]
5. 设置界面: 展示[未完待续...]
6. 排行榜界面: 展示[未完待续...]

数据设计层

```

1  // C/C++:
2  // ----- 数据设计 -----
3  /*
4      地图:
5          map[i][j][k]表示地图i中坐标(j,k)的值
6          0表示空地 格子
7          1表示遮挡物 格子
8          2表示玩家 格子
9          3表示目标 格子
10         如: map[4][3][6] = 1 表示地图4中(3,6)的位置是 遮挡物
11     */
12     int map[10][19][19];
13
14     /*
15         当前地图目标点:
16         (target[0], target[1], target[2]), 表示地图中的目标坐标
17     */
18     int target[3];
19
20 // ----- 数据设计 -----

```

服务 service 层

```
1 // ----- service -----
2 /*
3     负责人：张三
4     功能：init：初始化游戏数据
5         将地图的值初始化为 Level 关卡地图
6         玩家位置设置为地图上某点
7     参数：int Level：关卡数
8     返回值：void
9 */
10 void init(int Level);
11
12 /*
13     *难点1
14     负责人：张三
15     功能：iswin：根据传入的玩家当前坐标(map对应位置) 判断是否达到目标
16     参数：
17         x：当前玩家的x坐标
18         y：当前玩家的y坐标
19     返回值：
20         0表示没有获胜
21         1表示玩家胜利
22 */
23 int iswin(int x, int y);
24
25 /*
26     负责人：张三
27     功能：playerMove：玩家向指定方向移动一个格子
28         如果map[l][x][y]是空地 / 目标值 则修改map[l][x][y]的值为玩家对应值
29         并 修改玩家上一个位置为空地
30         否则 不操作
31     参数：
32         l：当前地图编号
33         x：当前回合玩家移动的x坐标
34         y：当前回合玩家移动的y坐标
35     返回值：
36         0表示移动失败(有遮挡物)
37         1表示移动成功
38
39 */
40 int playerMove(int l, int x, int y);
41 // ----- service -----
```

展示界面 view 层

```
1 // ----- view -----
2 /*
3     负责人：张三
4     功能：menuView：展示选项，玩家可以在这里选择
5         [选关界面] / [设置界面] / [排行榜界面] / 退出游戏
6         选关界面：调用选关界面函数levelview();
7         设置界面：敬请期待...
8         排行榜界面：敬请期待...
9         退出游戏：调用exit(0);
10    参数：void
```

```

11     返回值: void
12 */
13 void menuView();
14
15 /*
16     负责人: 张三
17     功能: levelView: 展示选项, 玩家可以在此选择
18         [游戏界面], 或者回到[开始界面]
19         游戏界面: 调用游戏界面函数mapView();
20         开始界面: 调用开始界面函数menuView();
21     参数: void
22     返回值: void
23 */
24 void levelView();
25
26 /*
27     负责人: 张三
28     功能: gameView_ShowMap: 根据map数组 打印地图
29     参数: void
30     返回值: void
31 */
32 void gameView_ShowMap();
33
34 /*
35     负责人: 张三
36     功能: winView: 将goal坐标和用户当前坐标进行匹配 打印游戏胜利界面
37         用户可以按任意键回到选关界面
38     参数: void
39     返回值: void
40 */
41 void winView();
42
43 /*
44     *难点2
45     负责人: 张三
46     功能: gameView: 游戏界面整合
47         初始化游戏数据(调用函数init())
48         while(1){
49             打印游戏界面(调用函数gameView_ShowMap())
50             接收玩家方向输入: 'w' 'a' 's' 'd'
51
52             移动(调用移动函数playerMove())
53             (如果移动失败 重新开始循环)
54
55             判断游戏是否胜利(调用胜利判断函数iswin())
56             (如果游戏胜利 调用胜利界面函数 然后结束当前界面)
57         }
58     参数: void
59     返回值: void
60 */
61 void gameView();
62 // ----- view -----

```

